Fast Programmable Networks & Encapsulated Protocols

...

David S. Miller
(Red Hat Inc.)

How Low Can You Go?

Facilities deep in the stack are stateful and feature-rich.

But you have to get there (and it wants state, lots of it).

Mechanisms built closer to the bottom of the stack are faster.

But often lack generality or portability.

This is, however, not a requirement.

Enter: XDP

What is XDP

Express Data Path

Built using eBPF for fast filtering and forwarding.

Executes directly in receive handler of the driver.

Executes before packet enters the stack.

Executes before SKB metadata is built and attached.

Completely generic.

Why Do We Need XDP?

We must have an alternative to DPDK.

Why?

Because DPDK is not Linux:

- Bypasses Linux stack instead of working in concert with it.
- Requires 3rd party modules and licensing.
- Defines it’s own security model for userspace accessing networking hardware.

XDP eBPF Packet Metadata

/usr/include/linux/bpf.h:

```
struct xdp_md {
        u32 data;
        u32 data_end;
};
```

XDP eBPF Actions

```
enum xdp_action {

        XDP_ABORTED = 0,
        XDP_DROP,
        XDP_PASS,
        XDP_TX,
};
```

XDP eBPF Result Codes

eBPF return code which determines what to do with the packet:

DROP - Drop packet and do not pass up to the stack.

ABORTED - eBPF program error, treat like DROP.

PASS - Accept packet and pass it up to the stack.

TX - Transmit packet out the same interface it arrived on,

do not pass up to the stack.

Example: DDoS Mitigation

eBPF map is created holding table of IP addresses to reject.

XDP eBPF program parses header to find the source IP address.

That source address is looked up in the map.

If found, eBPF program returns “DROP” code.

Upon “DROP”, packet buffer is returned to hardware and packet never enters stack.

Otherwise SKB metadata is attached and packet enters stack.

Example: Load Balancing

eBPF program has array of servers.

Hash over source address, full 4-tuple etc.

Computed hash indexes into array to choose server.

Server entry is used to rewrite destination IP and MAC address.

“TX” is returned by eBPF program.

Driver transmits rewritten packet out same interface it arrived on.

Restrictions

Each packet buffer must fit in a single page.

Facilitates the packet metadata given to eBPF programs.

XDP currently applies to all RX queues of device.

Transmit can only occur on the same device.

Future XDP Directions

Allow per-RX-queue XDP instances.

Chip uses filters to classify traffic to specific queues and thus specific XDP instances.

Allow forwarding to non-same interfaces.

Would allow real XDP based routing. Needs a port table.

Implementation could be complicated (DMA mapping, etc.)

Other XDP Uses...

ILA router

GRO assist

Traffic sampling

Customized statistics

Socket pre-demux

Encapsulated Protocols

Also known as: Transports over UDP

Also known as: Protocols in Userspace

Protocols such as TCP are encapsulated using GUE.

With optional encryption.

Since any userspace application can send UDP packets, any userspace entity can send GUE encapsulated TCP frames.

Therefore TCP stacks can be implemented in userspace.

ToU Motivation

Faster propagation of new protocol features.

(Android still doesn't support TCP Fast Open)

Eliminating middleware boxes from the equation.

(If all ToU flows are encrypted, analysis is nearly impossible)

Examples of ToU arguably already exist, such as QUIC (multiplexed stream protocol over UDP used in Google Chrome)

ToU Practical Matters

All GRO/GSO issues that exist with tunnels, exist with ToU

GSO seems like a solved problem with Partial Segmentation Offload

Trick is to tell the HW that the IP header is “big” and includes the GUE header.

GRO is non-trivial to solve... but perhaps doable.

Aggregation of ToU is difficult and we lack an ability to specify heuristics.

Other ToU Issues...

ECN is “interesting”...

Who influences the IP level ECN bits? Tunnel or embedded TCP stream?

Recent congestion control algorithms cannot be done in userspace ToU TCP.

They require interacting with FQ packet scheduler for proper pacing.

Timers and their granularity in userspace can be an issue.

“Packet still in qdisc/driver” issue.

Do We Want ToU?

It really causes us to open Pandora's Box.

So we better understand all of the implications before we deploy it.

Also, can we accept the fundamental restrictions (see previous slide)?

There really is no turning back if we made a bad choice here.